Contenido

- Introducción
- Cookies
- Cuando usar cookies
- Sesiones
- El ciclo de vida de una sesión
- Configuración avanzada de la cookie de sesión
- Seguridad en las sesiones

UD6: Persistencia de Datos en PHP

Introducción a la Persistencia Web

El protocolo HTTP, sobre el que se fundamenta la web, es un protocolo **sin estado** (stateless). Esto significa que cada petición que un navegador hace al servidor es independiente de las anteriores. El servidor no recuerda nada sobre el usuario de una página a otra.

Para crear aplicaciones web interactivas (como un carrito de la compra o un sistema de login), necesitamos una forma de "recordar" información del usuario a través de múltiples peticiones. Aquí es donde entran en juego las **cookies** y las **sesiones**.

Cookies: Almacenamiento en el Cliente

Una cookie es un pequeño fichero de texto que el servidor envía al navegador del usuario para que este lo guarde. En cada petición posterior al mismo dominio, el navegador devolverá automáticamente la cookie al servidor.

Crear una Cookie con `setcookie()`

Para crear una cookie en PHP, utilizamos la función `setcookie()`. Es **muy importante** llamarla antes de enviar cualquier otra salida al navegador (antes de cualquier etiqueta HTML, texto o incluso un espacio en blanco).

La sintaxis ha evolucionado. Aunque se pueden pasar los parámetros uno a uno, la práctica recomendada desde PHP 7.3 es usar un array de opciones, lo que hace el código más legible y mantenible.

```php
<?php
// Forrma tradicional (aún válida)
setcookie(
    string $name,
    string $value = "",
    int $expires_or_options = 0,
    string $path = "",
    string $domain = "",
    bool $secure = false,
    bool $httponly = false
);

// Forma moderna con array de opciones (RECOMENDADA)
setcookie(string $name, string $value = "", array $options = []);
?>
```

Opciones de la Cookie

- **`expires`**: Marca de tiempo Unix que indica cuándo caduca la cookie. Se suele calcular con `time()` más el número de segundos de vida. Si se omite o es 0, la cookie expira al cerrar el navegador.
- **`path`**: La ruta del servidor donde la cookie estará disponible. Con `'/'`, estará disponible en todo el dominio.
- **`domain`**: El dominio (o subdominio) para el que la cookie es válida.
- **`secure`**: Si se establece a `true`, la cookie solo se enviará sobre conexiones seguras HTTPS.
- **`httponly`**: Si es `true`, la cookie no será accesible mediante lenguajes de script del lado del cliente, como JavaScript. Es una medida de seguridad crucial para prevenir ataques XSS (Cross-Site Scripting).
- **`samesite`**: Una medida de seguridad vital contra ataques CSRF (Cross-Site Request Forgery). Puede tener los valores:
  - **`Strict`**: La cookie solo se envía en peticiones originadas desde el mismo sitio.
  - **`Lax`**: (Valor por defecto en PHP 8+) La cookie se envía con la navegación normal al sitio de origen, pero no en peticiones de otros sitios (p.ej. desde un formulario en otra web).
  - **`None`**: La cookie se envía en todas las peticiones, pero **requiere** que la opción `secure` sea `true`.

Ejemplo práctico

```php
<?php
// Cookie que dura 1 hora, disponible en todo el sitio,
// solo por HTTPS, no accesible por JS y con política SameSite estricta.
$nombre_cookie = 'usuario_preferencias';
$valor_cookie = json_encode(['tema' => 'oscuro', 'idioma' => 'es']);
$opciones = [
    'expires' => time() + 3600, // 1 hora
    'path' => '/',
    // 'domain' => '.tudominio.com', // Descomentar si es necesario
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
];

setcookie($nombre_cookie, $valor_cookie, $opciones);

// Cookie que se borra al cerrar el navegador
setcookie('visita_temporal', '1');
?>
<!DOCTYPE html>
<html>
<body>
    <h1>Página con Cookies</h1>
    <p>Se han configurado las cookies.</p>
</body>
</html>
```

Leer y Eliminar Cookies

Para acceder al valor de las cookies enviadas por el navegador, usamos el array superglobal `$_COOKIE`. Un detalle importante: una cookie creada en un script no estará disponible en `$_COOKIE` hasta la **siguiente petición** HTTP.

```php
<?php
// Comprobar si la cookie de preferencias existe
if (isset($_COOKIE['usuario_preferencias'])) {
    $preferencias = json_decode($_COOKIE['usuario_preferencias'], true);
    echo "Tema preferido: " . htmlspecialchars($preferencias['tema']);
} else {
    echo "No se encontraron preferencias de usuario.";
}

// Para eliminar una cookie, la creamos con una fecha de expiración en el pasado.
if (isset($_POST['borrar_preferencias'])) {
    setcookie('usuario_preferencias', '', time() - 3600, '/');
    // Redirigimos para que el cambio se vea reflejado inmediatamente
    header('Location: ' . $_SERVER['PHP_SELF']);
    exit();
}
?>
```

Cuándo Usar Cookies: Casos de Uso y Ejemplos

La regla de oro es: usa cookies para almacenar **información no sensible** que mejore la experiencia del usuario a lo largo de varias visitas, incluso después de que haya cerrado el navegador. Las cookies son ideales para "recordar" preferencias o estados que no comprometen la seguridad.

A diferencia de las sesiones, que son para datos temporales y sensibles (como el estado de autenticación), las cookies son para la **persistencia a largo plazo**de datos de personalización.

1. Almacenar Preferencias del Usuario

Este es el uso más común y recomendado. Permite que tu sitio web se adapte a las elecciones del usuario cada vez que te visita, creando una experiencia más personal y agradable .

- Preferencia de idioma.
- Tema del sitio (claro/oscuro).
- Configuraciones de accesibilidad (tamaño de fuente, contraste).
- Número de resultados a mostrar por página en una lista.

Ejemplo: Selector de Tema

Guardamos la elección del tema del usuario en una cookie que dura un año.

```php
<?php
// Procesar la elección del usuario enviada por un formulario
if (isset($_POST['tema'])) {
    $tema_elegido = $_POST['tema'] === 'oscuro' ? 'oscuro' : 'claro';

    // Crear la cookie con duración de 1 año (365 días)
    $opciones = [
        'expires' => time() + (365 * 24 * 60 * 60),
        'path' => '/',
        'secure' => true, // Siempre que sea posible
        'httponly' => true,
        'samesite' => 'Lax'
    ];
    setcookie('preferencia_tema', $tema_elegido, $opciones);

    // Redirigir para aplicar el cambio inmediatamente
    header('Location: ' . $_SERVER['PHP_SELF']);
    exit();
}

// Leer la cookie para aplicar el tema
$tema_actual = $_COOKIE['preferencia_tema'] ?? 'claro'; // 'claro' es el valor por defecto
?>
<!DOCTYPE html>
<html lang="es" class="<?= htmlspecialchars($tema_actual) ?>">
<!-- El resto del HTML usaría la clase 'claro' u 'oscuro' para estilizar -->
<body>
    <h1>Mi Sitio Web</h1>
    <p>Actualmente estás usando el tema <?= htmlspecialchars($tema_actual) ?>.</p>
    <form method="POST" action="">
        <label for="tema">Elige un tema:</label>
        <select name="tema" id="tema" onchange="this.form.submit()">
            <option value="claro" <?= $tema_actual === 'claro' ? 'selected' : '' ?>>Claro</option>
            <option value="oscuro" <?= $tema_actual === 'oscuro' ? 'selected' : '' ?>>Oscuro</option>
        </select>
    </form>
</body>
</html>
```

2. Funcionalidad "Recordarme" en el Login

Para evitar que un usuario tenga que iniciar sesión cada vez que visita tu sitio desde su dispositivo de confianza, puedes usar una cookie. **¡Atención!** Nunca almacenes la contraseña en una cookie. En su lugar, guarda un token aleatorio y seguro.

Ejemplo: Creación de Token "Recordarme"

Cuando el usuario marca "Recordarme", generamos un token, lo guardamos en la base de datos asociado a su ID y también en una cookie.

```php
<?php
// Tras un login exitoso...
if ($login_exitoso && isset($_POST['recordarme'])) {
    $id_usuario = 123; // ID del usuario que se logueó

    // 1. Generar un token seguro
    $selector = bin2hex(random_bytes(16));
    $validador = bin2hex(random_bytes(32));
    $token_cookie = $selector . ':' . $validador;

    // 2. Guardar el selector y el hash del validador en la BD
    $validador_hashed = hash('sha256', $validador);
    $fecha_expiracion = time() + (30 * 24 * 60 * 60); // 30 días
    // $db->prepare("INSERT INTO tokens_recordarme (selector, validador_hashed, id_usuario, expira) VALUES (?, ?, ?, ?)")
    //   ->execute([$selector, $validador_hashed, $id_usuario, date('Y-m-d H:i:s', $fecha_expiracion)]);

    // 3. Enviar la cookie al usuario
    setcookie('recordarme', $token_cookie, [
        'expires' => $fecha_expiracion,
        'path' => '/',
        'secure' => true,
        'httponly' => true,
        'samesite' => 'Strict'
    ]);
}
?>
```

En visitas posteriores, si la cookie `recordarme` existe, se comprueba el token contra la base de datos para autenticar al usuario automáticamente. Este es un patrón de diseño avanzado pero muy seguro.

3. Seguimiento Simple y Mensajes Únicos

Las cookies son útiles para realizar acciones una sola vez por visitante, como mostrar un banner de bienvenida, un aviso sobre la política de cookies, o para un seguimiento anónimo básico.

Ejemplo: Mostrar un Aviso de Bienvenida una sola vez

```php
<?php
$mostrar_aviso = false;
if (!isset($_COOKIE['visitante_recurrente'])) {
    // Es la primera visita (en mucho tiempo), así que mostramos el aviso
    $mostrar_aviso = true;

    // Creamos la cookie para no volver a mostrarlo por un año
    setcookie('visitante_recurrente', '1', [
        'expires' => time() + (365 * 24 * 60 * 60),
        'path' => '/',
        'samesite' => 'Lax'
    ]);
}
?>
<!DOCTYPE html>
<html>
<body>
    <?php if ($mostrar_aviso): ?>
        <div style="background: #eef; padding: 1rem; text-align: center;">
            <h2>¡Bienvenido por primera vez a nuestro sitio!</h2>
            <p>Esperamos que disfrutes de tu visita.</p>
        </div>
    <?php endif; ?>

    <h1>Contenido principal de la página</h1>
</body>
</html>
```

Sesiones: Persistencia en el Servidor

Las sesiones son un mecanismo para almacenar información del usuario en el **servidor**. A diferencia de las cookies, en el cliente solo se guarda un identificador único de sesión (normalmente en una cookie llamada `PHPSESSID`). Esto es más seguro porque los datos sensibles nunca abandonan el servidor.

El Ciclo de Vida de una Sesión

El trabajo con sesiones sigue un ciclo sencillo:

1. Iniciar o reanudar una sesión

Se usa la función `session_start()`. Al igual que `setcookie()`, debe ser llamada **antes de cualquier salida** al navegador. Esta función comprueba si existe un ID de sesión válido; si lo hay, carga los datos de la sesión; si no, crea una nueva.

Es buena práctica comprobar el estado de la sesión antes de intentar iniciarla para evitar errores:

```php
<?php
// Iniciar sesión solo si no hay una activa
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
?>
```

2. Almacenar y acceder a datos

Los datos de la sesión se gestionan a través del array superglobal `$_SESSION`. Podemos leer, escribir y eliminar variables como en cualquier otro array.

```php
<?php
// (Después de session_start())

// Guardar datos en la sesión
$_SESSION['id_usuario'] = 123;
$_SESSION['nombre_usuario'] = 'daw_user';
$_SESSION['hora_login'] = time();

// Leer datos
$nombre = $_SESSION['nombre_usuario'] ?? 'Invitado';
echo "Bienvenido, " . htmlspecialchars($nombre);

// Eliminar una variable de sesión específica
unset($_SESSION['hora_login']);
?>
```

3. Destruir la sesión

Cuando el usuario cierra sesión (logout), debemos destruir toda la información de la sesión. El proceso completo y seguro es el siguiente:

```php
<?php
session_start();

// 1. Eliminar todas las variables de sesión
$_SESSION = [];

// 2. Borrar la cookie de sesión
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

// 3. Finalmente, destruir la sesión del servidor
session_destroy();

echo "Sesión cerrada correctamente.";
// Redirigir al usuario a la página de inicio
// header('Location: /index.php');
// exit();
?>
```

Simplemente usar `session_destroy()` no es suficiente, ya que no borra las variables del array `$_SESSION` actual ni la cookie del cliente. El método anterior es el más robusto.

Configuración Avanzada de la Cookie de Sesión

Aunque PHP gestiona la cookie de sesión (normalmente `PHPSESSID`) de forma automática, tenemos control total sobre sus propiedades. Configurar correctamente esta cookie es un paso **fundamental para la seguridad** de la aplicación, especialmente para mitigar ataques como el secuestro de sesión (Session Hijacking) y la fijación de sesión (Session Fixation).

Para ello, disponemos de dos funciones principales: `session_set_cookie_params()` para establecer los parámetros y `session_get_cookie_params()` para consultarlos.

Establecer Parámetros con `session_set_cookie_params()`

Esta función nos permite definir cómo se comportará la cookie de sesión que PHP envía al navegador. La regla más importante es que **debe ser llamada antes de `session_start()`** para que tenga efecto.

Al igual que `setcookie()`, la forma moderna y recomendada (desde PHP 7.3) es pasar las opciones en un array asociativo. Esto hace que el código sea mucho más claro y seguro.

```php
<?php
// Definimos un tiempo de vida para la cookie de sesión de 30 minutos (1800 segundos)
$lifetime = 1800;

// Establecemos los parámetros de la cookie de sesión de forma segura
session_set_cookie_params([
    'lifetime' => $lifetime,
    'path' => '/',
    'domain' => 'tudominio.com', // Cambiar por tu dominio
    'secure' => true,      // La cookie solo se enviará sobre HTTPS
    'httponly' => true,  // No accesible por JavaScript
    'samesite' => 'Lax'    // 'Strict' o 'Lax'. Lax es un buen equilibrio de seguridad y usabilidad.
]);

// Ahora sí, iniciamos la sesión. PHP usará la configuración anterior.
session_start();

// ... el resto de la lógica de tu aplicación ...
?>
```

¡Importante!
Llamar a `session_set_cookie_params()` después de `session_start()` no producirá un error, pero **no tendrá ningún efecto** en la sesión actual, ya que las cabeceras de la cookie ya habrán sido enviadas o leídas.

Obtener Parámetros con `session_get_cookie_params()`

Esta función devuelve un array asociativo con los parámetros de la cookie de sesión. Es especialmente útil cuando necesitamos asegurarnos de que estamos eliminando la cookie correcta del navegador del cliente, ya que nos proporciona la ruta, dominio y otras propiedades con las que fue configurada.

Un detalle clave es que esta función devuelve los parámetros **tal y como están definidos en el fichero `php.ini`** o como fueron establecidos por `session_set_cookie_params()` en el script actual [12, 17]. No lee los valores de la cookie que el navegador ha enviado.

Caso Práctico: Inicio y Destrucción Segura de una Sesión

Combinando ambas funciones, podemos crear un ciclo de vida de sesión robusto y seguro. El siguiente ejemplo muestra cómo configurar la sesión al inicio y cómo destruirla por completo al final.

```php
<?php
// --- INICIO SEGURO DE LA SESIÓN (ej. en un fichero `init.php`) ---

// Solo si no hay una sesión activa
if (session_status() === PHP_SESSION_NONE) {
    // Configuramos la cookie para que sea más segura
    session_set_cookie_params([
        'lifetime' => 1800,       // 30 minutos de vida
        'path' => '/',
        'secure' => true,         // Solo HTTPS
        'httponly' => true,     // No accesible por JS
        'samesite' => 'Lax'
    ]);

    session_start();
}

// Después de un login exitoso, siempre regeneramos el ID
// session_regenerate_id(true);
?>
```

```php
<?php
// --- DESTRUCCIÓN COMPLETA DE LA SESIÓN (ej. en `logout.php`) ---

// Reanudamos la sesión para poder acceder a ella
//Si estamos en el logout.php no nos importa que no haya sesión iniciada, si no la hay se inicia, y si ya hay, se continúa.
session_start();

// 1. Borramos todas las variables de la sesión
$_SESSION = array();

// 2. Obtenemos los parámetros de la cookie de sesión para poder borrarla
$params = session_get_cookie_params();

// 3. Borramos la cookie del cliente enviando una con fecha de expiración pasada
setcookie(
    session_name(), // Nombre de la cookie de sesión (p.ej. 'PHPSESSID')
    '',            // Valor vacío
```